ICT Academy of Kerala · Python Programming

Dictionary Case Studies for Data Science

Step-by-Step Solutions · Trainer Copy

Python Session 4 · Four classroom case studies
All outputs were produced by running the code with Python 3.13.
Not for distribution to participants.

How to use this pack

Give the students the student PDF. Walk around while they work, then show the solution step by step. For each task, show the code, run it, and ask the class to explain each line before you move on.

#	Case study	Key ideas
1	Dataset Profiler	dictionary of lists (columns), None for missing values, counting, summary statistics
2	Data Cleaning and Encoding	list of dictionaries (rows), missing-value imputation, label encoding with a dictionary, grouping
3	Customer Review Analysis	word frequency, stop words, a sentiment lexicon dictionary, sorting (count, word) tuples, labels
4	Sales GroupBy Report	grouping and aggregating with dictionaries, averages, finding the best, nested dictionary (pivot), tuple sorting

Suggested order: the four case studies get harder. For a shorter class, use 1 and 4. For a full practice session, use all four.

Case Study 1: Dataset Profiler

File: `dataset_profiler.py`

> **Data science link.** This is what `df.shape`, `df.isna().sum()`, `df.describe()` and `df['city'].value_counts()` do in pandas. Today you build them by hand so you understand what happens underneath.

Task 1: Create the dataset

```
data = {
    "student_id": [101, 102, 103, 104, 105, 106, 107, 108],
    "age":        [21, 22, None, 20, 23, 21, 22, 24],
    "hours":      [5.5, 3.0, 7.0, 2.5, 6.0, 4.5, None, 8.0],
    "city":       ["Kochi", "Kollam", "Kochi", "Thrissur", "Kollam", "Kochi", "Kollam",
"Kochi"],
    "score":      [78, 65, 88, 52, 74, 70, 69, 91],
}
```

A table stored as 'dictionary of lists' is column-oriented. `pandas.DataFrame(data)` accepts exactly this shape.

Task 2: Shape of the data

```
rows = len(data["student_id"])
print("DATASET PROFILE")
print("=" * 40)
print("Rows:", rows)
print("Columns:", len(data))
print("Column names:", ", ".join(data))
```

OUTPUT

```
DATASET PROFILE
========================================
Rows: 8
Columns: 5
Column names: student_id, age, hours, city, score
```

Looping over or joining a dictionary uses its **keys**, so `", ".join(data)` joins the column names.

Task 3: Missing values per column

```
print()
print("MISSING VALUES")
for col, values in data.items():
    missing = 0
    for v in values:
        if v is None:
            missing += 1
    print(f"{col:<12}{missing:>3}")
```

OUTPUT

```
MISSING VALUES
student_id    0
age           1
hours         1
city          0
score         0
```

Always test for missing values with `is None`. A nested loop: the outer one walks the columns, the inner one the values.

Task 4: Numeric summary

```
print()
print("NUMERIC SUMMARY")
print(f"{'Column':<8}{'Count':>6}{'Mean':>8}{'Min':>7}{'Max':>7}")
for col in ["age", "hours", "score"]:
    valid = []
    for v in data[col]:
        if v is not None:
            valid.append(v)
    print(f"{col:<8}{len(valid):>6}{sum(valid) / len(valid):>8.2f}{min(valid):>7}
{max(valid):>7}")
```

OUTPUT

```
NUMERIC SUMMARY
Column   Count    Mean    Min    Max
age          7   21.86     20     24
hours        7    5.21    2.5    8.0
score        8   73.38     52     91
```

Build a list of the valid values first, then use `sum`, `len`, `min` and `max` on it. Dividing by `len(data[col])` would be wrong because it counts the missing values.

Task 5: Category frequency

```
print()
print("CITY FREQUENCY")
city_counts = {}
for city in data["city"]:
    city_counts[city] = city_counts.get(city, 0) + 1
for city, n in city_counts.items():
    print(f"{city:<10}{n:>2}  {'#' * n}")
```

OUTPUT

```
CITY FREQUENCY
Kochi      4  ####
Kollam     3  ###
Thrissur   1  #
```

This is `value_counts()`: the counting pattern with `get(key, 0) + 1`.

Challenge

Convert the column-oriented `data` into a **list of row dictionaries**, one dictionary per student (the shape pandas calls `to_dict('records')`). Use a loop over `range(rows)` and an inner loop over the columns. Print the first two rows.

```
records = []
for i in range(rows):
    row = {}
    for col in data:
        row[col] = data[col][i]
    records.append(row)
print()
print("FIRST TWO ROWS")
print(records[0])
print(records[1])
```

OUTPUT

```
FIRST TWO ROWS
{'student_id': 101, 'age': 21, 'hours': 5.5, 'city': 'Kochi', 'score': 78}
{'student_id': 102, 'age': 22, 'hours': 3.0, 'city': 'Kollam', 'score': 65}
```

Common mistake to show

Adding up a column that still contains None:

```
ages = [21, 22, None, 20]
print(sum(ages))
```

OUTPUT

```
Traceback (most recent call last):
  File "demo.py", line 2, in <module>
    print(sum(ages))
          ~~~^^^^^^
TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

Discussion questions

- Why do we divide by the number of **valid** values and not by the number of rows?
- What is the difference between a dictionary of lists (columns) and a list of dictionaries (rows)? When would each be handy?

Complete program

```
data = {
    "student_id": [101, 102, 103, 104, 105, 106, 107, 108],
    "age":        [21, 22, None, 20, 23, 21, 22, 24],
    "hours":      [5.5, 3.0, 7.0, 2.5, 6.0, 4.5, None, 8.0],
    "city":       ["Kochi", "Kollam", "Kochi", "Thrissur", "Kollam", "Kochi", "Kollam",
"Kochi"],
    "score":      [78, 65, 88, 52, 74, 70, 69, 91],
}
rows = len(data["student_id"])
print("DATASET PROFILE")
print("=" * 40)
print("Rows:", rows)
print("Columns:", len(data))
print("Column names:", ", ".join(data))
print()
print("MISSING VALUES")
for col, values in data.items():
    missing = 0
    for v in values:
        if v is None:
            missing += 1
    print(f"{col:<12}{missing:>3}")
print()
print("NUMERIC SUMMARY")
print(f"{'Column':<8}{'Count':>6}{'Mean':>8}{'Min':>7}{'Max':>7}")
for col in ["age", "hours", "score"]:
    valid = []
    for v in data[col]:
        if v is not None:
            valid.append(v)
    print(f"{col:<8}{len(valid):>6}{sum(valid) / len(valid):>8.2f}{min(valid):>7}
{max(valid):>7}")
print()
print("CITY FREQUENCY")
city_counts = {}
for city in data["city"]:
    city_counts[city] = city_counts.get(city, 0) + 1
for city, n in city_counts.items():
    print(f"{city:<10}{n:>2}  {'#' * n}")
records = []
for i in range(rows):
    row = {}
    for col in data:
        row[col] = data[col][i]
    records.append(row)
print()
print("FIRST TWO ROWS")
print(records[0])
print(records[1])
```

Case Study 2: Data Cleaning and Encoding

File: `data_cleaning.py`

> **Data science link.** This is `fillna(mean)`, `map()` / `LabelEncoder` and `groupby()` in pandas. Machine learning models need numbers without gaps, so cleaning and encoding come before any model.

Task 1: Create the records

```
records = [
    {"id": 1, "plan": "Basic",    "usage": 12.5, "churned": "No"},
    {"id": 2, "plan": "Premium",  "usage": None, "churned": "Yes"},
    {"id": 3, "plan": "Standard", "usage": 8.0,  "churned": "No"},
    {"id": 4, "plan": "Basic",    "usage": 15.0, "churned": "Yes"},
    {"id": 5, "plan": "Premium",  "usage": 22.5, "churned": "No"},
    {"id": 6, "plan": "Standard", "usage": None, "churned": "No"},
    {"id": 7, "plan": "Basic",    "usage": 9.5,  "churned": "Yes"},
    {"id": 8, "plan": "Premium",  "usage": 20.0, "churned": "No"},
]
```

A list of dictionaries is the 'row' view of a table: one dictionary per row.

Task 2: Check the data

```
print("CHECK THE DATA")
missing = 0
total = 0
for r in records:
    if r["usage"] is None:
        missing += 1
    else:
        total += r["usage"]
mean_usage = round(total / (len(records) - missing), 2)
print("Missing usage values:", missing)
print("Mean usage (valid values):", mean_usage)
```

OUTPUT

```
CHECK THE DATA
Missing usage values: 2
Mean usage (valid values): 14.58
```

The mean is computed from the six valid values only: 87.5 / 6.

Task 3: Fill the gaps (imputation)

```
for r in records:
    if r["usage"] is None:
        r["usage"] = mean_usage
print()
print("CLEANED DATA")
print(f"{'ID':<4}{'Plan':<10}{'Usage':>7}{'Churned':>9}")
for r in records:
    print(f"{r['id']:<4}{r['plan']:<10}{r['usage']:>7.2f}{r['churned']:>9}")
```

OUTPUT

```
CLEANED DATA
ID  Plan        Usage  Churned
1   Basic       12.50       No
2   Premium     14.58      Yes
3   Standard     8.00       No
4   Basic       15.00      Yes
5   Premium     22.50       No
6   Standard    14.58       No
7   Basic        9.50      Yes
8   Premium     20.00       No
```

Changing `r["usage"]` changes the dictionary inside the list, because `r` refers to the same dictionary.

Task 4: Encode the categories

```
plan_codes = {"Basic": 0, "Standard": 1, "Premium": 2}
churn_codes = {"No": 0, "Yes": 1}
for r in records:
    r["plan_code"] = plan_codes[r["plan"]]
    r["churn_flag"] = churn_codes[r["churned"]]
print()
print("ENCODED DATA")
print(f"{'ID':<4}{'plan_code':>10}{'usage':>8}{'churn_flag':>12}")
for r in records:
    print(f"{r['id']:<4}{r['plan_code']:>10}{r['usage']:>8.2f}{r['churn_flag']:>12}")
```

OUTPUT

```
ENCODED DATA
ID   plan_code   usage  churn_flag
1            0   12.50           0
2            2   14.58           1
3            1    8.00           0
4            0   15.00           1
5            2   22.50           0
6            1   14.58           0
7            0    9.50           1
8            2   20.00           0
```

A dictionary is a lookup table, which makes it a natural encoder: text in, number out.

Task 5: Churn rate by plan

```
churned_total = 0
by_plan = {}
for r in records:
    churned_total += r["churn_flag"]
    if r["plan"] not in by_plan:
        by_plan[r["plan"]] = [0, 0]
    by_plan[r["plan"]][0] += 1
    by_plan[r["plan"]][1] += r["churn_flag"]
print()
print(f"Overall churn rate: {churned_total / len(records) * 100:.1f}%")
print("CHURN RATE BY PLAN")
for plan in sorted(by_plan):
    count, lost = by_plan[plan]
    print(f"{plan:<10}{lost}/{count}  {lost / count * 100:>5.1f}%")
```

OUTPUT

```
Overall churn rate: 37.5%
CHURN RATE BY PLAN
Basic     2/3   66.7%
Premium   1/3   33.3%
Standard  0/2    0.0%
```

Because the churn flag is 0 or 1, adding the flags counts the churned customers. This is a `groupby` with a mean. What does the result tell the company about Basic customers?

Challenge

Writing the plan codes by hand does not scale to 50 categories. Write a function `build_encoder(values)` that takes a list of category values and **returns** a dictionary giving each unique value a code (0, 1, 2 ...) in alphabetical order. Collect the plans from the records into a list, call your function and print the result.

```
def build_encoder(values):
    codes = {}
    for v in sorted(values):
        if v not in codes:
            codes[v] = len(codes)
    return codes

plans = []
for r in records:
    plans.append(r["plan"])
print()
print("AUTO ENCODER")
print(build_encoder(plans))
```

OUTPUT

```
AUTO ENCODER
{'Basic': 0, 'Premium': 1, 'Standard': 2}
```

Common mistake to show

Encoding a category that is not in the dictionary:

```
plan_codes = {"Basic": 0, "Standard": 1, "Premium": 2}
print(plan_codes["Gold"])
```

OUTPUT

```
Traceback (most recent call last):
  File "demo.py", line 2, in <module>
    print(plan_codes["Gold"])
          ~~~~~~~~~~^^^^^^^^
KeyError: 'Gold'
```

Discussion questions

- Filling gaps with the mean is a simple choice. What else could you use? (The median, or the most common value.) When would the mean mislead?
- Why is 'Basic = 0, Standard = 1, Premium = 2' acceptable for a plan, but not for a city name?

Complete program

```
records = [
    {"id": 1, "plan": "Basic",    "usage": 12.5, "churned": "No"},
    {"id": 2, "plan": "Premium",  "usage": None, "churned": "Yes"},
    {"id": 3, "plan": "Standard", "usage": 8.0,  "churned": "No"},
    {"id": 4, "plan": "Basic",    "usage": 15.0, "churned": "Yes"},
    {"id": 5, "plan": "Premium",  "usage": 22.5, "churned": "No"},
    {"id": 6, "plan": "Standard", "usage": None, "churned": "No"},
    {"id": 7, "plan": "Basic",    "usage": 9.5,  "churned": "Yes"},
    {"id": 8, "plan": "Premium",  "usage": 20.0, "churned": "No"},
]
print("CHECK THE DATA")
missing = 0
total = 0
for r in records:
    if r["usage"] is None:
        missing += 1
    else:
        total += r["usage"]
mean_usage = round(total / (len(records) - missing), 2)
print("Missing usage values:", missing)
print("Mean usage (valid values):", mean_usage)
for r in records:
    if r["usage"] is None:
        r["usage"] = mean_usage
print()
print("CLEANED DATA")
print(f"{'ID':<4}{'Plan':<10}{'Usage':>7}{'Churned':>9}")
for r in records:
    print(f"{r['id']:<4}{r['plan']:<10}{r['usage']:>7.2f}{r['churned']:>9}")
plan_codes = {"Basic": 0, "Standard": 1, "Premium": 2}
churn_codes = {"No": 0, "Yes": 1}
for r in records:
    r["plan_code"] = plan_codes[r["plan"]]
    r["churn_flag"] = churn_codes[r["churned"]]
print()
print("ENCODED DATA")
print(f"{'ID':<4}{'plan_code':>10}{'usage':>8}{'churn_flag':>12}")
for r in records:
    print(f"{r['id']:<4}{r['plan_code']:>10}{r['usage']:>8.2f}{r['churn_flag']:>12}")
churned_total = 0
by_plan = {}
for r in records:
    churned_total += r["churn_flag"]
    if r["plan"] not in by_plan:
        by_plan[r["plan"]] = [0, 0]
    by_plan[r["plan"]][0] += 1
    by_plan[r["plan"]][1] += r["churn_flag"]
print()
print(f"Overall churn rate: {churned_total / len(records) * 100:.1f}%")
print("CHURN RATE BY PLAN")
for plan in sorted(by_plan):
    count, lost = by_plan[plan]
    print(f"{plan:<10}{lost}/{count}  {lost / count * 100:>5.1f}%")
def build_encoder(values):
    codes = {}
    for v in sorted(values):
        if v not in codes:
            codes[v] = len(codes)
    return codes

plans = []
for r in records:
    plans.append(r["plan"])
print()
```

```
print("AUTO ENCODER")
print(build_encoder(plans))
```

Case Study 3: Customer Review Analysis

File: `review_analysis.py`

> **Data science link.** This is the start of **Natural Language Processing**: word counts like `collections.Counter` or `CountVectorizer`, and a lexicon-based sentiment score. The same ideas power review analysis at large companies.

Task 1: Create the data

```
reviews = [
    "The delivery was fast and the product is great",
    "Terrible service and the product arrived broken",
    "Good quality but the delivery was late",
    "I love this product and the support team is excellent",
    "Poor packaging and slow delivery",
    "Great product and fast delivery very happy",
]
stop_words = ["the", "and", "is", "was", "but", "i", "this", "very", "a"]
lexicon = {"good": 1, "great": 2, "excellent": 3, "love": 2, "fast": 1, "happy": 2,
           "slow": -1, "late": -1, "poor": -2, "bad": -2, "broken": -2, "terrible": -3}
```

Stop words are common words that carry little meaning. The lexicon maps a word to a score: positive for good words, negative for bad ones.

Task 2: Top 5 words

```
counts = {}
for review in reviews:
    for word in review.lower().split():
        if word not in stop_words:
            counts[word] = counts.get(word, 0) + 1
pairs = []
for word, n in counts.items():
    pairs.append((n, word))
pairs.sort(reverse=True)
print("TOP 5 WORDS")
print("-" * 20)
for n, word in pairs[:5]:
    print(f"{word:<12}{n:>3}")
```

OUTPUT

```
TOP 5 WORDS
--------------------
product       4
delivery      4
great         2
fast          2
terrible      1
```

A list of tuples sorts by the first item, then the second. Putting the count first sorts by count. Words with equal counts are ordered by the word, Z to A, because of `reverse=True`.

Task 3: Sentiment score of each review

```
print()
print("SENTIMENT PER REVIEW")
print("-" * 20)
scores = []
labels = []
for i, review in enumerate(reviews, start=1):
    score = 0
    for word in review.lower().split():
        score += lexicon.get(word, 0)
    if score > 0:
        label = "Positive"
    elif score < 0:
        label = "Negative"
    else:
        label = "Neutral"
    scores.append(score)
    labels.append(label)
    print(f"Review {i}: {score:>3}  {label}")
```

OUTPUT

```
SENTIMENT PER REVIEW
--------------------
Review 1:   3  Positive
Review 2:  -5  Negative
Review 3:   0  Neutral
Review 4:   5  Positive
Review 5:  -3  Negative
Review 6:   5  Positive
```

`enumerate(reviews, start=1)` numbers the reviews from 1. The lexicon dictionary does all the 'knowledge' work.

Task 4: Summary

```
label_count = {}
for label in labels:
    label_count[label] = label_count.get(label, 0) + 1
print()
print("SUMMARY")
for label in sorted(label_count):
    print(f"{label:<10}{label_count[label]}")
print(f"Average score: {sum(scores) / len(scores):.2f}")
```

OUTPUT

```
SUMMARY
Negative  2
Neutral   1
Positive  3
Average score: 0.83
```

The counting pattern again, this time on labels instead of words.

Challenge

Which words appear in **negative reviews only**? Using the reviews, labels and stop words, collect the words that appear in Negative reviews and the words that appear in Positive reviews (skip stop words). Print the heading `WORDS ONLY IN NEGATIVE REVIEWS` and then those words, sorted and joined by a comma and a space.

```
positive_words = {}
negative_words = {}
for i in range(len(reviews)):
    for word in reviews[i].lower().split():
        if word in stop_words:
            continue
        if labels[i] == "Positive":
            positive_words[word] = True
        elif labels[i] == "Negative":
            negative_words[word] = True
only_negative = []
for word in negative_words:
    if word not in positive_words:
        only_negative.append(word)
print()
print("WORDS ONLY IN NEGATIVE REVIEWS")
print(", ".join(sorted(only_negative)))
```

OUTPUT

```
WORDS ONLY IN NEGATIVE REVIEWS
arrived, broken, packaging, poor, service, slow, terrible
```

Common mistake to show

Forgetting to convert to lowercase: 'Great' is not 'great':

```
lexicon = {"great": 2}
print(lexicon.get("Great", 0))
print(lexicon.get("Great".lower(), 0))
```

OUTPUT

```
0
2
```

Discussion questions

- Review 3 scored 0 (Neutral): 'Good quality but the delivery was late'. Is a lexicon score fair here? What does it miss?
- What would 'not good' score? Why do modern models need more than a word list?

Complete program

```
reviews = [
    "The delivery was fast and the product is great",
    "Terrible service and the product arrived broken",
    "Good quality but the delivery was late",
    "I love this product and the support team is excellent",
    "Poor packaging and slow delivery",
    "Great product and fast delivery very happy",
]
stop_words = ["the", "and", "is", "was", "but", "i", "this", "very", "a"]
lexicon = {"good": 1, "great": 2, "excellent": 3, "love": 2, "fast": 1, "happy": 2,
           "slow": -1, "late": -1, "poor": -2, "bad": -2, "broken": -2, "terrible": -3}
counts = {}
for review in reviews:
    for word in review.lower().split():
        if word not in stop_words:
            counts[word] = counts.get(word, 0) + 1
pairs = []
for word, n in counts.items():
    pairs.append((n, word))
pairs.sort(reverse=True)
print("TOP 5 WORDS")
print("-" * 20)
for n, word in pairs[:5]:
    print(f"{word:<12}{n:>3}")
print()
print("SENTIMENT PER REVIEW")
print("-" * 20)
scores = []
labels = []
for i, review in enumerate(reviews, start=1):
    score = 0
    for word in review.lower().split():
        score += lexicon.get(word, 0)
    if score > 0:
        label = "Positive"
    elif score < 0:
        label = "Negative"
    else:
        label = "Neutral"
    scores.append(score)
    labels.append(label)
    print(f"Review {i}: {score:>3}  {label}")
label_count = {}
for label in labels:
    label_count[label] = label_count.get(label, 0) + 1
print()
print("SUMMARY")
for label in sorted(label_count):
    print(f"{label:<10}{label_count[label]}")
print(f"Average score: {sum(scores) / len(scores):.2f}")
positive_words = {}
negative_words = {}
for i in range(len(reviews)):
    for word in reviews[i].lower().split():
        if word in stop_words:
            continue
        if labels[i] == "Positive":
            positive_words[word] = True
        elif labels[i] == "Negative":
            negative_words[word] = True
only_negative = []
for word in negative_words:
    if word not in positive_words:
        only_negative.append(word)
print()
```

```
print("WORDS ONLY IN NEGATIVE REVIEWS")
print(", ".join(sorted(only_negative)))
```

Case Study 4: Sales GroupBy Report

File: `sales_report.py`

> **Data science link.** This is `df.groupby('region')['amount'].sum()`, `.mean()` and `pivot_table()` in pandas. Group-by is the most used operation in data analysis.

Task 1: Create the sales data

```
sales = [
    {"region": "North", "product": "Laptop", "amount": 55000},
    {"region": "South", "product": "Phone",  "amount": 18000},
    {"region": "North", "product": "Phone",  "amount": 22000},
    {"region": "East",  "product": "Laptop", "amount": 48000},
    {"region": "South", "product": "Laptop", "amount": 52000},
    {"region": "West",  "product": "Phone",  "amount": 15000},
    {"region": "North", "product": "Laptop", "amount": 60000},
    {"region": "East",  "product": "Phone",  "amount": 20000},
    {"region": "South", "product": "Phone",  "amount": 25000},
    {"region": "West",  "product": "Laptop", "amount": 45000},
]
```

Each dictionary is a row; `region`, `product` and `amount` are the columns.

Task 2: Total sales by region

```
totals = {}
for s in sales:
    totals[s["region"]] = totals.get(s["region"], 0) + s["amount"]
print("TOTAL SALES BY REGION")
print("-" * 22)
for region in sorted(totals):
    print(f"{region:<8}{totals[region]:>10,}")
```

OUTPUT

```
TOTAL SALES BY REGION
----------------------
East        68,000
North      137,000
South       95,000
West        60,000
```

This is the group-and-add pattern: the key is the group, the value is the running total.

Task 3: Average sale by region

```
counts = {}
for s in sales:
    counts[s["region"]] = counts.get(s["region"], 0) + 1
print()
print("AVERAGE SALE BY REGION")
print("-" * 22)
for region in sorted(totals):
    print(f"{region:<8}{totals[region] / counts[region]:>12,.2f}")
```

OUTPUT

```
AVERAGE SALE BY REGION
----------------------
East       34,000.00
North      45,666.67
South      31,666.67
West       30,000.00
```

Average = total divided by count. Two dictionaries with the same keys work together.

Task 4: Best region

```
best_region = ""
best_total = 0
for region, total in totals.items():
    if total > best_total:
        best_region = region
        best_total = total
print()
print(f"Best region: {best_region} ({best_total:,})")
```

OUTPUT

```
Best region: North (137,000)
```

The 'best so far' pattern, applied to the pairs of a dictionary.

Task 5: Region by product table

```
pivot = {}
for s in sales:
    region = s["region"]
    product = s["product"]
    if region not in pivot:
        pivot[region] = {}
    pivot[region][product] = pivot[region].get(product, 0) + s["amount"]
print()
print("REGION x PRODUCT")
print(f"{'Region':<8}{'Laptop':>10}{'Phone':>10}")
for region in sorted(pivot):
    print(f"{region:<8}{pivot[region].get('Laptop', 0):>10,}{pivot[region].get('Phone',
0):>10,}")
```

OUTPUT

```
REGION x PRODUCT
Region      Laptop     Phone
East        48,000    20,000
North      115,000    22,000
South       52,000    43,000
West        45,000    15,000
```

A pivot table is a dictionary of dictionaries: the outer key is the row, the inner key is the column.

Challenge

Rank the regions by total sales, highest first, and print each region's **share of the grand total** as a percentage with one decimal. Hint: build a list of `(total, region)` tuples and sort it with `sort(reverse=True)`. Print the heading `REGION SHARE OF SALES`, then the region (width 8) and the share (width 6) followed by `%`.

```
grand = sum(totals.values())
ranking = []
for region, total in totals.items():
    ranking.append((total, region))
ranking.sort(reverse=True)
print()
print("REGION SHARE OF SALES")
for total, region in ranking:
    print(f"{region:<8}{total / grand * 100:>6.1f}%")
```

OUTPUT

```
REGION SHARE OF SALES
North     38.1%
South     26.4%
East      18.9%
West      16.7%
```

Common mistake to show

Adding to a region that has not been seen yet:

```
totals = {}
totals["North"] += 55000
```

OUTPUT

```
Traceback (most recent call last):
  File "demo.py", line 2, in <module>
    totals["North"] += 55000
    ~~~~~~^^^^^^^^^
KeyError: 'North'
```

Discussion questions

- Which two patterns (counting, group-and-add) did you combine to get the average?
- How would you answer 'which product sells best in each region?' using the pivot dictionary?

Complete program

```
sales = [
    {"region": "North", "product": "Laptop", "amount": 55000},
    {"region": "South", "product": "Phone",  "amount": 18000},
    {"region": "North", "product": "Phone",  "amount": 22000},
    {"region": "East",  "product": "Laptop", "amount": 48000},
    {"region": "South", "product": "Laptop", "amount": 52000},
    {"region": "West",  "product": "Phone",  "amount": 15000},
    {"region": "North", "product": "Laptop", "amount": 60000},
    {"region": "East",  "product": "Phone",  "amount": 20000},
    {"region": "South", "product": "Phone",  "amount": 25000},
    {"region": "West",  "product": "Laptop", "amount": 45000},
]
totals = {}
for s in sales:
    totals[s["region"]] = totals.get(s["region"], 0) + s["amount"]
print("TOTAL SALES BY REGION")
print("-" * 22)
for region in sorted(totals):
    print(f"{region:<8}{totals[region]:>10,}")
counts = {}
for s in sales:
    counts[s["region"]] = counts.get(s["region"], 0) + 1
print()
print("AVERAGE SALE BY REGION")
print("-" * 22)
for region in sorted(totals):
    print(f"{region:<8}{totals[region] / counts[region]:>12,.2f}")
best_region = ""
best_total = 0
for region, total in totals.items():
    if total > best_total:
        best_region = region
        best_total = total
print()
print(f"Best region: {best_region} ({best_total:,})")
pivot = {}
for s in sales:
    region = s["region"]
    product = s["product"]
    if region not in pivot:
        pivot[region] = {}
    pivot[region][product] = pivot[region].get(product, 0) + s["amount"]
print()
print("REGION x PRODUCT")
print(f"{'Region':<8}{'Laptop':>10}{'Phone':>10}")
for region in sorted(pivot):
    print(f"{region:<8}{pivot[region].get('Laptop', 0):>10,}{pivot[region].get('Phone',
0):>10,}")
grand = sum(totals.values())
ranking = []
for region, total in totals.items():
    ranking.append((total, region))
ranking.sort(reverse=True)
print()
print("REGION SHARE OF SALES")
for total, region in ranking:
    print(f"{region:<8}{total / grand * 100:>6.1f}%")
```